

C Programming Tutorial

“Strings” in C

A string in C is a one dimensional array of characters terminated by a null character `\0`.

```
#include <stdio.h>
```

```
int main (){
```

```
    char str1[6] = {'H','e','l','l','o','\0'};  
    printf("%s\n",str1);
```

Create an array of characters terminated by a null character.

```
    char str2[]="hello";  
    printf("%s\n",str2);
```

Create an array of characters using a string constant. Terminating character will be added automatically.

```
    char *str3 = "hello";  
    printf("%s\n",str3);
```

Create an array of characters using the pointer notation.

```
    return 0;█
```

```
}
```

User Input

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
int main() {
    int ch;
    char name[50];
```

```
    printf("Enter your name\n");
    if(fgets(name, sizeof(name), stdin) == NULL)
        return 1;
```

```
    if(name[strlen(name) - 1] != '\n') {
        printf("Input is too long\n");
        while(((ch = getchar()) != '\n') && (ch != EOF));
    }
```

```
    else {
        strtok(name, "\n");
    }
    printf("Your name is %s", name);
```

```
    return 0;
```

```
}
```

Scan a string from the user

If input is too long, fgets()
doesn't add a trailing new line

Read the rest of characters
from stdin (flush stdin)

Remove trailing new line added
by fgets()

User Input

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {

    char buff[4];
    int age;
    printf("Enter your age:\n");

    if(fgets(buff, sizeof(buff), stdin) == NULL) → Scan a string from the user
        return 1;

    /* ... validate input (new line, length, content ... */

    age = atoi(buff); → Convert String to integer

    printf("Your age is %d\n", age);
    return 0;
}
```

Some C functions

`sizeof (x)`: returns the number of bytes occupied by x

`sizeof(int)` : returns the number of bytes of an int which is 4 bytes in the 32-bit architecture

`int a[10]; sizeof(a) = 10*sizeof(int)`

`strlen(str)`: returns the actual size of the str

`strtok(str, "\n")`: removes the trailing new line

`atoi`: array to integer

Exercise

- 1) Write a C program that declares a character array of size 10, then fills it from the user with a string, your program should calculate the size of the input string.
- 2) Write a C program to remove the last occurrence of a character in a string. Read the string size, string and character from the command line
- 3) Write a C program that counts the number of occurrences of each letter in a string.

```
#include <stdio.h>
//#include <stdlib.h>
#include <string.h>

int strlen (char[], int);
int main (){

    char str[10];
    if(fgets ( str, sizeof(str), stdin) == NULL)
        return 1;
    if(str [strlen (str) - 1] != '\n'){
        printf("Input is too long\n");
    }

    printf("str is %d" , strlen(str));
;

    return 0;
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char **argv){
    if (argc!=4)
        printf ("Enter string size, string and a character only");
    else{
        int size = atoi(argv[1]);
        char string[size];
        int i =0;
        for (i=0; i<strlen(argv[2]);i++)
            string[i] = argv[2][i];
        string[i]='\0';
        char c = argv[3][0];
        printf ("your values are as follows:\n\t size =%d\n\tstring = %s\n$
        int j = 0;
        int index = -1;
        for (j=0; j< strlen(string) ; j++)
            if(string[j]==c)
                index = j;
        if(index != -1){
            i = index;
            while(i<strlen(string)){
                string[i]=string[i+1];
                i++;
            }
        }
        printf("%s",string);
    }
}

```

```

return 0 ;

```



```
#include <stdio.h>
#include <string.h>

int main (int argc ,char **argv){
    if(argc!=2)
        printf("error");
    else{
        int i;
        int count[26]={0};
        for (i = 0 ; i < strlen (argv[1]) ; i++)
            if (argv[1][i] >= 'a' && argv[1][i] <= 'z')
                count[argv[1][i] - 'a']++;

        for (i = 0; i < 26; i++)
            printf("%c occurs %d times\n", i + 'a', count[i]);
    }
    return 0;
}
```



Quiz

Write a program that scan a string from the user and print ("I am kidding you don't have a Quiz for this time :P ")

Write a C program that reads a string using fgets and an integer using scanf and print them

Write a C program that prints a string "hello" repeatedly excluding the first char each time. For example, "abcde" will be printed as such: "abcde", "bcde", "cde", "de", "e".

Revision

Write a C program that reads a string (one word) from the **command line** and counts the number of occurrences of each letter in it.

Assume characters are only the 26 small caps (a-z)

```
#include <stdio.h>
#include <string.h>

int main (int argc ,char **argv){
    if(argc!=2)
        printf("error");
    else{
        int i;
        int count[26]={0};
        for (i = 0 ; i < strlen (argv[1]) ; i++)
            if (argv[1][i] >= 'a' && argv[1][i] <= 'z')
                count[argv[1][i] - 'a']++;

        for (i = 0; i < 26; i++)
            printf("%c occurs %d times\n", i + 'a', count[i]);
    }
    return 0;
}
```

Solution

Structures

A Struct is a way to compose existing types into structure. Structures provide a way of storing many different values in variables of potentially different types under the same name.

Makes the program more modular.

Makes the program more compact and easier to modify.

In generally, structs are useful whenever a lot of data needs to be grouped together. For example, Store information about an address book.

Defining a structure: **struct struct_name name_of_a_single_struct;**

Accessing variable of the structure: **name_of_a_single_struct.name_of_variable;**

C Structure Syntax

```
#include <stdio.h>

struct person{
    int age;
    float salary;
};

int main(){

    struct person bob;

    bob.age = 25;
    bob.salary = 900.5;

    printf("%s%d %.2f\n", "person bob: ", bob.age, bob.salary);

    struct person *bob_junior;
    bob_junior = &bob;

    printf("%s%d %.2f\n", "person bob_junior", bob_junior->age, bob_junior->sallary);
    return 0;
}
```

Defining a structure with 2 members.

Create a single structure.

Access the elements of the structure.

Define a pointer to a structure.

Point to the memory address of the "bob" structure.

Use the "." operator when dealing with a structure. Use "->" operator when dealing with a pointer to a structure.

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct {
    int year;
    int month;
```

g/cbootcamp/8_Pointers.html

```
    int day;
} date;
```

```
int main(void) {
```

```
    date *today;
    today = malloc(sizeof(date));
```

```
// the explicit way of accessing fields of our struct
(*today).day = 15;
(*today).month = 6;
(*today).year = 2012;
```

```
// the more readable shorthand way of doing it
today->day = 15;
today->month = 6;
today->year = 2012;
```

```
printf("the date is %d/%d/%d\n", today->day, today->month, today->year);
```

```
free(today);
```

```
return 0;
```

```
}
```

Pointer to a struct notations

C Typedef

The C keyword “typedef” is used to give a type a new name.

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef unsigned char BYTE;
```

Define a term BYTE for one byte.

```
typedef struct persons{
```

```
    int age;
    char *name;
```

```
}Person;
```

Define a term Person for the structure.

```
int main(){
```

```
    BYTE a = 'a';
    printf("%c\n", a);
```

Use the new term BYTE instead of “unsigned char”.

```
    Person bob;
    bob.age = 25;
    bob.name = "bob";
    printf("%s %d\n", bob.name, bob.age);
    return 0;
```

Use the new term Person to declare a structure instead of the previous notation we looked at.

```
}■
```


Exercise

Write a C program that scans numbers from the user representing x and y coordinates of 2 points on an x-y axis. Create a Struct called “Point” to store the values. Create a function that takes 2 Point arguments by reference to check whether 2 points are equal or no

Note: if you want to use Boolean include `<stdbool.h>`

Declaring Boolean in c is for example
`bool x = true;`

```

#include <stdio.h>
// #include <stdlib.h>
#include <stdbool.h>
typedef struct p {
    int x;
    int y;
} Point;
bool check(Point*, Point*);
int main(){
    Point p1, p2;
    scanf("%d%d%d%d", &(p1.x), &(p1.y), &(p2.x), &(p2.y));
    if(check(&p1, &p2)) printf("True, same point");
    else printf("False, not same point");
    return 0;
}
bool check(Point *a, Point *b){
    if(a->x==b->x && a->y==b->y)
        return true;
    return false;
}

```

Exercise - Solution

Time in C

Print the number of hours since EPOCH time

```
#include <stdio.h>
#include <time.h>

int main ()
{
    time_t sec;
    sec = time (NULL);

    printf ("Number of hours since January 1, 1970 is %ld \n",
            sec/3600);
    return 0;
}
```

Time in C

Print the current local time

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    time_t mytime;
    mytime = time(NULL);
    printf(ctime(&mytime));

    return 0;
}
```

Time in C

Calculate the elapsed time of a function

```
#include <stdio.h>
#include <time.h>

int main(void)
{
    time_t start, end;
    long counter;

    start = time(NULL);

    for(counter = 0; counter < 800000000; counter++);

    end = time(NULL);
    printf("The loop took %f seconds.\n",
        difftime(end, start));
    return 0;
}
```

More practice

1. Write a C program that finds the maximum and minimum elements of an array of integers of size 10.
2. Write a C program that creates an array of “points” (defined in the above exercise) of size *n* (where *n* is a command line argument). Fill the array from the user. Create a function that takes the array of Points and size as parameters and checks if it includes 2 equal points.
3. Write a C program that searches for a number in an array of integers using the following function’s signature: *bool search(int A[], int num, int size)*.
4. Write a C program that scans a string and a character from the user and returns true whether the string contains the character and false otherwise. Use the following function’s signature: *bool contains(char*, char)*.

```
#include <stdio.h>
```

```
int main (){
```

```
    int arr[10]={4,5,6,20,7,5,-4,7,-10,1};
```

```
    int max = arr[0];
```

```
    int min = arr[0];
```

```
    int i = 0;
```

```
    for (i = 1; i < 10 ;i++)
```

```
{
```

```
        if (arr[i] > max)
```

```
            max = arr[i];
```

```
        else if(arr[i] < min)
```

```
            min = arr[i];
```

```
}
```

```
    printf("min = %d\nmax = %d\n",min, max);
```

```
    return 0;
```

```
}
```

Solution 1

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
typedef struct p {
    int x;
    int y;
}Point;
bool contains(Point[], int);
int main(int argc, char **argv){
    //do the error checking ...
    int n = atoi(argv[1]);
    Point p[n];
    int i;
    for (i=0;i<n;i++)
        scanf("%d%d",&(p[i].x),&(p[i].y));
    if(contains(p,n))
        printf("TRUE");
    else
        printf("FALSE");
    return 0;
}

```

```

bool contains(Point a[], int s){
    int i =0,j=0;
    for(i=0;i<s-1;i++)
        for(j=i+1;j<s;j++)
            if(a[i].x==a[j].x && a[i].y==a[j].y)
                return true;
    return false;
}

```

Solution 2


```

#include <stdio.h>
#include <stdbool.h>
bool search(int[],int, int);
int main(){
    int size;
    printf("Enter the size of the array:\n");
    scanf("%d",&size);
    int arr[size];
    int i =0;
    printf("Enter the elements of the array\n");
    for (i=0 ; i<size;i++)
        scanf("%d",&arr[i]);
    printf("Enter the number you are searching for\n");
    int num ;
    scanf("%d", &num);
    if (search(arr, num, size))
        printf("FOUND\n");
    else printf("NOT FOUND\n");
}
bool search(int a[], int num, int size){
    int i = 0;
    for (i=0;i<size;i++)
        if(a[i]==num)
            return true;
    return false;
}

```

Solution 3